Playwright + TestNG Interview Questions

Oct 5, 2026

104 questions on Playwright for Java and TestNG, each answered with an explanation, a short *Example* and an **In the framework** link to the file in playwright-framework that does it (or why it is not used there).

The project tests the Sauce Demo shop through the browser and the Restful Booker API directly, on Chromium, Firefox and WebKit: 54 tests in a normal run, 58 with the demo tests that fail and skip on purpose. Sections 1–10 are concept questions, section 11 is about this project, sections 12–13 are coding tasks with working code.

1. Playwright basics

Q1. What is Playwright?
Playwright is Microsoft's open-source library for automating Chromium, Firefox and WebKit with one API. It has official bindings for Java, JavaScript/TypeScript, Python and .NET, and also sends plain HTTP requests, so UI and API tests can share one tool.
Example:
`Playwright.create().chromium().launch().newPage().navigate("https://www.saucedemo.com")`.
In the framework: UI tests drive Sauce Demo and API tests call Restful Booker, both through Playwright — BrowserManager.java, ApiClient.java.

Q2. How does Playwright talk to the browser?
The Java library starts a bundled Node.js driver process and sends commands to it over a pipe; the driver controls each browser through its own protocol (Chrome DevTools Protocol for Chromium, patched protocols for Firefox and WebKit). There is no WebDriver server and no separate browser driver to install.
Example: the first `Playwright.create()` on a machine unpacks the driver; `install` downloads the matching browser builds.
In the framework: `mvn exec:java -Dexec.args="install"` runs Playwright's CLI through the exec plugin in pom.xml.

Q3. Which browsers does Playwright support?
Chromium (also Chrome and Edge channels), Firefox and WebKit (the engine of Safari), each in a version pinned to the Playwright release. WebKit runs on Windows and Linux too, so Safari-like behaviour can be tested without a Mac.
Example: `playwright.webkit().launch()`.
In the framework: `-Dbrowser=chromium|firefox|webkit` picks the engine in `BrowserManager.browserType()`; CI runs all three — tests.yml.

Q4. Playwright vs Selenium: the main differences?
Playwright auto-waits for elements and retries assertions, isolates tests in cheap browser contexts, controls the network (mocking, blocking), records traces and videos, and ships its own browsers. Selenium follows the W3C WebDriver standard, supports more languages and real branded browsers, and has Grid for remote runs.
Example: Selenium needs `WebDriverWait` + `ExpectedConditions`; Playwright's `assertThat(locator).hasText("Products")` waits by itself.
In the framework: no explicit wait exists in any test; see the class comment of BasePage.java.

Q5. What are Playwright, Browser, BrowserContext and Page?
`Playwright` is the entry point (driver connection). `Browser` is one launched browser process. `BrowserContext` is an isolated, incognito-like session with its own cookies and storage. `Page` is one tab in a context.
Example: `browser.newContext().newPage()`.
In the framework: one Playwright + Browser per thread, a new context + page per UI test — BrowserManager.java.

Q6. Headless vs headed, and slowMo?
Headless runs without a window (faster, used in CI); headed shows the browser for debugging. `slowMo` pauses after every action so a person can follow it.
Example: `new BrowserType.LaunchOptions().setHeadless(false).setSlowMo(300)`.
In the framework: `headless` and `slowMo` keys in config.properties; `mvn clean test -Dheadless=false -DslowMo=300`.

Q7. How are the browsers installed?
With the Playwright CLI: `install` downloads all browsers, `install chromium` one, `install --with-deps` also the Linux system libraries. Builds go to a shared cache (`ms-playwright`).
Example: `mvn exec:java -e -D exec.mainClass=com.microsoft.playwright.CLI -D exec.args="install"`.
In the framework: the exec plugin's `mainClass` is preset in pom.xml; CI uses `install --with-deps <browser>` in tests.yml.

Q8. Is Playwright thread-safe in Java?
No. A `Playwright` object and everything created from it must be used from one thread. For parallel tests each thread needs its own `Playwright` instance.
Example: `ThreadLocal<Playwright> PLAYWRIGHT = new ThreadLocal<>()`.
In the framework: `ThreadLocal` fields for Playwright, Browser, BrowserContext and Page in BrowserManager.java, with Surefire `parallel=classes`, 3 threads.

2. Locators

Q9. What is a Locator, and how is it different from an ElementHandle?
A `Locator` is a lazy description of how to find elements; it is resolved again on every action, so it never goes stale and it waits for the element. An `ElementHandle` points at one DOM node found once and can go stale; Playwright discourages it.
Example: `Locator button = page.getByTestId("login-button"); button.click();` — nothing is searched until `click()`.
In the framework: pages keep `Locator` fields created in the constructor, e.g. LoginPage.java; no `ElementHandle` is used.

Q10. Which locators does Playwright recommend?
User-facing ones first: `getByRole` (role + accessible name), `getByLabel`, `getByPlaceholder`, `getByText`, `getByAltText`, `getByTitle`, then `getByTestId` for a stable contract with developers. CSS and XPath work but break more easily with layout changes.
Example: `page.getByRole(AriaRole.BUTTON, new Page.GetByRoleOptions().setName("Login"))`.
In the framework: test ids for most elements, role for buttons (`Add to cart`, `Open Menu`) — ShopPage.java, InventoryPage.java.

Q11. How does getByTestId work, and how do you change its attribute?
By default it matches `data-testid`.
`playwright.selectors().setTestIdAttribute("data-test")` switches it to another attribute for every page of that Playwright.
Example: Sauce Demo uses `data-test="login-button"`, so `page.getByTestId("login-button")`.
In the framework: set once per thread in `BrowserManager.playwright()` — BrowserManager.java.

Q12. Can you still use CSS and XPath?
Yes, through `page.locator(...)`: a string starting with `//` or `..` is XPath, otherwise CSS (or a prefixed engine such as `text=` or `xpath=`). Playwright's CSS also pierces open shadow DOM.
Example: `page.locator("#user-name")`, `page.locator("//input[@name='user-name']")`.
In the framework: the five-strategy comparison in `locatorStrategiesFindTheSameField` — PlaywrightScenariosTest.java; `[data-test$='title-link']` CSS in InventoryPage.java.

Q13. What is strict mode?
An action on a locator that matches more than one element throws a strict mode violation instead of picking the first. It forces precise locators. To act on one of many, say which: `first()`, `last()`, `nth(i)` or a `filter`.

Example: `page.getByTestId("inventory-item").click()` fails (6 matches); `.first().click()` works.
In the framework: WebKit once hit a strict-mode violation (old list's 6 names still on screen); fixed with `waitUntilLoaded()` — ProductPage.java.

Q14. How do you find an element inside a specific container (filter and chaining)?
Chain locators (`card.getByRole(...)`) to search inside another match, and use `filter(new Locator.FilterOptions().setHas(...))` or `setHasText(...)` to keep only the containers that contain something.
Example: the product card that contains the exact name, then its button.
In the framework: `InventoryPage.card(name)` + `getByRole(BUTTON, "Add to cart")` — InventoryPage.java.

Q15. What does locator.or() do?
`a.or(b)` matches elements matching either locator, which lets one wait cover two possible outcomes. Combine with `first()` to stay strict when both could show.
Example: wait for the product title or the login error, whichever appears first.
In the framework: `inventory.title().or(error).first().waitFor()` in `LoginPage.loginAs` , then a clear `IllegalStateException` for a refused login — LoginPage.java.

Q16. How do you read text from many elements?
`allInnerTexts()` returns the visible text of every match (`allTextContents()` the raw text); `all()` returns one locator per match for further work. These do not retry, so wait for the page first.
Example: `page.getByTestId("inventory-item-name").allInnerTexts()` .
In the framework: `BasePage.texts(locator)` and `InventoryPage.imageSources()` — BasePage.java, InventoryPage.java.

Q17. How do you find locators quickly?
With codegen (`playwright codegen <url>` records actions and suggests locators) or the Playwright Inspector's pick-locator tool (`PWDEBUG=1`). Both prefer role and test-id locators.
Example: `mvn exec:java -Dexec.args="codegen https://www.saucedemo.com"` .
In the framework: not stored in code; the CLI is wired through the exec plugin in pom.xml, so codegen runs from the project folder.

3. Auto-waiting and assertions

Q18. What is auto-waiting?
Before an action Playwright waits until the element is actionable: attached, visible, stable (not animating), enabled, and receiving events (not covered). `fill` also waits for it to be

editable. Only after the timeout does the action fail.
Example: `page.getByTestId("checkout").click()` waits until the button can really be clicked.
In the framework: there is no sleep or explicit wait in any page object — CartPage.java.

Q19. What are web-first assertions?
`PlaywrightAssertions.assertThat(locator|page|response)` checks retry until the condition holds or the timeout ends, so a check written right after an action does not race the page. The error shows expected, received and a call log.
Example: `assertThat(page.getByTestId("title")).hasText("Products")`.
In the framework: used in every UI test, e.g. LoginTest.java; the demo failure shows its message — ReportDemoTest.java.

Q20. assertThat(...).hasText() vs assertEquals(locator.innerText(), ...)?
`innerText()` reads once, at that instant; if the page is still updating the TestNG assertion fails. `hasText` retries. Use plain reads only for values that are already settled (after a wait) or to compute something.
Example: `assertThat(cartBadge).hasText("2")` vs `assertEquals(cartBadge.innerText(), "2")`.
In the framework: UI state uses `assertThat`; maths on settled pages uses `assertEquals` after `waitUntilLoaded()` — CheckoutTest.java, CheckoutOverviewPage.java.

Q21. Which assertions are used most?
On a locator: `isVisible`, `isHidden`, `hasText` (also an array for lists), `containsText`, `hasCount`, `hasAttribute`, `hasValue`, `isEnabled`, `isChecked`. On a page: `hasURL`, `hasTitle`. Any can be negated with `not()`.
Example: `assertThat(cart.items()).hasText(new String[] {"Sauce Labs Backpack", "Sauce Labs Bike Light"})`.
In the framework: `hasText(String[])` in CartTest.java; `not().hasURL(...)` in LoginTest.java.

Q22. Which timeouts exist and how are they set?
Action timeout (`context.setDefaultTimeout`, default 30 s), navigation timeout (`setDefaultNavigationTimeout`), assertion timeout (`PlaywrightAssertions.setDefaultAssertionTimeout`, default 5 s), plus a per-call option such as `new Locator.ClickOptions().setTimeout(...)`.
Example: `context.setDefaultTimeout(10_000)`.
In the framework: one `timeoutMillis` = 10 s for actions and assertions — BrowserManager.java, config.properties.

Q23. When do you still wait explicitly?
When nothing is acted on or asserted yet but the next step reads values, or when a single-page app changes the URL before drawing the page. Wait for an element unique to the

target page (`locator.waitFor()` or an assertion), or for an event: `waitForResponse` , `waitForURL` , `waitForLoadState` . Never `Thread.sleep` .
Example: `assertThat(page.getByTestId("back-to-products")).isVisible()` .
In the framework: every page has `waitUntilLoaded()` , called by each navigation method — ProductPage.java, InventoryPage.java.

Q24. Why is Thread.sleep bad in Playwright tests?
A fixed sleep is too long on a fast run (slow suite) and too short on a slow one (flaky test). Playwright's waits end as soon as the condition holds.
Example: replace `Thread.sleep(3000)` with `assertThat(locator).isVisible()` .
In the framework: no `Thread.sleep` anywhere in the project.

Q25. Can soft assertions be used with Playwright?
Playwright for Java has no built-in soft `assertThat` (JavaScript has `expect.soft`). Use TestNG's `SoftAssert` for plain values, or collect results and assert once.
Example: `SoftAssert soft = new SoftAssert(); soft.assertEquals(...); soft.assertAll();` .
In the framework: `softAssertChecksEverything` — TestNgFeaturesTest.java.

4. Browser, context and page

Q26. Why a new BrowserContext per test instead of a new browser?
A context is a fresh, isolated profile (cookies, local storage, cache) created in milliseconds inside a running browser; launching a browser takes far longer. Tests stay independent without paying the launch cost each time.
Example: `Browser` launched once per thread, `browser.newContext()` per test.
In the framework: `BrowserManager.startTest()` per UI test, `browser()` reused per thread — BrowserManager.java; CartTest.java relies on an empty cart at every start.

Q27. Which options can a context have?
Base URL, viewport, locale, timezone, geolocation and permissions, user agent, device emulation (`isMobile` , `hasTouch`), HTTP credentials, extra headers, storage state, video recording, offline mode, colour scheme.
Example: `new Browser.NewContextOptions().setBaseURL(url).setViewportSize(1440, 900)` .
In the framework: base URL, viewport 1440×900 and video directory — BrowserManager.java.

Q28. What does setBaseURL give you?
`page.navigate("/inventory.html")` and request paths resolve against it, so environments change in one setting and page objects hold only paths.

Example: base `https://www.saucedemo.com` + `navigate("/")`.
In the framework: `LoginPage.open()` navigates to `/`, `InventoryPage.PATH = "/inventory.html"` — LoginPage.java, InventoryPage.java.

Q29. How do you work with several tabs or a popup?
All pages of one context share its cookies. `context.newPage()` opens another tab; a tab opened by the app is caught with `page.waitForPopup(() -> link.click())` or `context.waitForPage(...)`. Each `Page` is driven directly, no window switching.
Example: `Page second = page.context().newPage(); second.navigate("/inventory.html");`.
In the framework: `secondTabSharesTheLogin` — PlaywrightScenariosTest.java.

Q30. How do you log in once and reuse it?
After logging in, `context.storageState()` (or `setPath(...)` to a file) saves cookies and local storage; a new context created with `setStorageState(...)` starts logged in. Suites log in once in setup and skip the login form in every test.
Example: `browser.newContext(new Browser.NewContextOptions().setStorageState(state))`.
In the framework: `savedLoginSkipsTheLoginForm` shows it; regular tests log in through the form on purpose, because login is part of what they check — PlaywrightScenariosTest.java.

Q31. How do you emulate a mobile device?
With context options: viewport, `setIsMobile(true)`, `setHasTouch(true)`, a mobile user agent and `deviceScaleFactor`. JavaScript has a device list (`devices['iPhone 13']`); in Java the values are set by hand. Firefox does not support `isMobile`.
Example: `new Browser.NewContextOptions().setViewportSize(390, 844).setIsMobile(true).setHasTouch(true)`.
In the framework: not used; Sauce Demo has no separate mobile layout worth testing, and the suite runs on Firefox too.

Q32. How do you handle frames?
`page.frameLocator("iframe#pay").getByRole(...)` (or `locator(...).contentFrame()`) finds elements inside an iframe, with the same auto-waiting. No switching back is needed.
Example: `page.frameLocator("#card").getByPlaceholder("Card number").fill("4242...")`.
In the framework: not used; Sauce Demo has no iframes.

Q33. How do you close everything properly?
Close the context after each test (this also finishes its video), and close each `Playwright` at the end (closing its browsers and driver). Not closing leaks browser processes.

Example: `context.close(); playwright.close();`.
In the framework: `finishTest()` closes the context in `UiTest.closePage`; `closeAll()` runs in `@AfterSuite` — UiTest.java, PlaywrightTest.java.

5. Network and API testing

Q34. How do you intercept requests?
`page.route(urlPattern, handler)` (or `context.route`) catches matching requests; the handler can `route.abort()`, `route.fulfill(...)` with a fake answer, `route.fallback()` or `route.resume()` (optionally with changed headers or body). Glob patterns, regexes and predicates are allowed.
Example: `page.route("**/*.{png,jpg,jpeg,svg}", Route::abort)`.
In the framework: `shopWorksWithImagesBlocked` — NetworkTest.java.

Q35. How do you mock a backend response?
`route.fulfill(new Route.FulfillOptions().setStatus(200).setContentType("application/json").setBody(json))` answers without the server, so a UI can be tested with empty lists, errors or edge values that the real backend cannot easily produce. `route.fetch()` + modify + `fulfill` patches a real answer.
Example: answer `/booking/1` with `{"firstname":"Mocked"}`.
In the framework: `mockedResponseReplacesTheServers` — NetworkTest.java.

Q36. How do you listen to all network traffic?
Page events: `onRequest`, `onResponse`, `onRequestFinished`, `onRequestFailed`. Handlers run on Playwright's thread, so collect into thread-safe lists and filter out third-party hosts and requests the browser cancelled itself.
Example: `page.onResponse(r -> { if (r.status() >= 400) failures.add(r.url()); })`.
In the framework: `noFailingRequestsWhileShopping` with `CopyOnWriteArrayList` and `cancelledByBrowser(...)` (Firefox reports `NS_BINDING_ABORTED`) — NetworkTest.java.

Q37. How do you wait for a specific response caused by an action?
`page.waitForResponse(predicate, () -> action)` starts listening before the action runs (no race) and returns the first matching `Response`. `waitForRequest` works the same way for requests.
Example: `Response image = page.waitForResponse(r -> r.url().endsWith(".jpg"), () -> login.loginAs("standard_user"))`.
In the framework: `loginLoadsTheProductImages` — PlaywrightScenariosTest.java.

Q38. How does Playwright test REST APIs?

`playwright.request().newContext(options)` creates an `APIRequestContext` with a base URL and default headers; `get`, `post`, `put`, `patch`, `delete` or `fetch(path, RequestOptions)` send requests and return an `APIResponse` (`status()`, `text()`, `headers()`). No browser is started.
Example: `request.fetch("/booking", RequestOptions.create().setMethod("POST").setData(json))`.
In the framework: ApiClient.java wraps it; endpoints in BookingApi.java.

Q39. How do you (de)serialise JSON for API tests?

Playwright sends a `String` or `byte[]` body (or a Map/object it serialises itself); for typed checks use Jackson or Gson. Java records map directly and compare by value, so a whole response can be checked in one `assertEquals`.
Example: `assertEquals(read.as(Booking.class), booking)`.
In the framework: records `Booking`, `BookingDates`, `CreatedBooking` + Json.java; BookingApiTest.java.

Q40. Does a failing HTTP status throw?

No, by default `APIResponse` is returned for 4xx/5xx too (`setFailOnStatusCode(true)` changes that). Returning it lets negative tests check 403 or 404 directly.
Example: `assertEquals(bookings.get(id).status(), 404)`.
In the framework: `changesNeedAToken` (403), `deletedBookingIsGone` (404) — BookingApiTest.java.

Q41. How do you handle auth tokens and cookies in API tests?

Get the token once per test (or per suite) and send it as a header or cookie; an `APIRequestContext` also keeps cookies from responses. Restful Booker wants `Cookie: token=<value>` for PUT, PATCH and DELETE.
Example: `options.setHeader("Cookie", "token=" + token)`.
In the framework: `BookingApi.token(...)` and the cookie header — BookingApi.java; `adminToken()` in ApiTest.java.

Q42. How can API calls speed up UI tests?

Create test data through the API instead of the UI, or call `page.request()` (it shares the browser context's cookies) to check URLs found on the page. Only the behaviour under test goes through the UI.
Example: `page.request().get(baseUrl + src).ok()` for every product image.
In the framework: `productImagesAreNotBrokenExceptForTheProblemUser` — PlaywrightScenariosTest.java; API tests create bookings with `givenBooking(...)`.

6. Debugging: traces, videos, screenshots

Q43. What is a Playwright trace?
A zip that records every action with before/after DOM snapshots, screenshots, network requests, console messages and the source line. The Trace Viewer replays it step by step, so a CI failure can be studied as if it ran locally.
Example: `context.tracing().start(new Tracing.StartOptions().setScreenshots(true).setSnapshots(true))`, then `stop(new Tracing.StopOptions().setPath(zip))`.
In the framework: started per test in `BrowserManager.startTest`, saved only for failures in `finishTest` — BrowserManager.java.

Q44. How do you open a trace?
`playwright show-trace trace.zip` opens the Trace Viewer locally; or drop the zip on trace.playwright.dev (it runs in the browser, the file is not uploaded).
Example: `mvn exec:java -Dexec.args="show-trace target/artifacts/<file>-trace.zip"`.
In the framework: the report prints this command and a link under every failed UI test — HtmlReportListener.java.

Q45. How is video recorded?
Set `setRecordVideoDir(dir)` (and optionally `setRecordVideoSize`) on the context; every page records a webm. The file is complete only after the context closes; then `page.video().saveAs(path)` copies it and `delete()` removes the temporary file.
Example: `Video v = page.video(); context.close(); v.saveAs(target); v.delete();`.
In the framework: recorded to `target/video-temp`, kept in `target/artifacts` for failures only — BrowserManager.java.

Q46. Page vs element screenshots?
`page.screenshot()` captures the viewport (or `setFullPage(true)` the whole page); `locator.screenshot()` captures one element. Both return PNG bytes and can also write a file with `setPath`.
Example: `page.locator("form").screenshot()`.
In the framework: full-page on failure in `UiTest.attachScreenshot` — UiTest.java; element in `LoginPage.formScreenshot` — LoginPage.java.

Q47. Why keep artifacts only for failed tests?
Traces and videos of passing tests take disk space and CI upload time but are rarely opened. Recording always and keeping on failure gives full evidence of the first failure without rerunning (the JS runner calls it `retain-on-failure`).

Example: `finishTest(fileName, failed)` saves or discards.
In the framework: `UiTest.closePage` passes `result.getStatus() == FAILURE` — UiTest.java.

Q48. How do you debug a test step by step?
Run headed with `slowMo`, or set `PWDEBUG=1` to open the Playwright Inspector, which pauses before each action, highlights locators and lets you step. `page.pause()` stops at a chosen line.
Example: `$env:PWDEBUG=1; mvn test -Dtest=LoginTest`.
In the framework: `-Dheadless=false -DslowMo=300` documented in pom.xml and the README; no `page.pause()` is left in the code.

Q49. What makes Playwright tests flaky, and how do you fix it?
Common causes: reading values before the page settled, shared state between tests, fixed sleeps, tests depending on order, third-party or slow demo servers. Fixes: web-first assertions, a fresh context per test, unique test data, `waitForResponse` for async work, and retries only for known external instability.
Example: WebKit read the old product list right after the URL changed; waiting for an element unique to the new page fixed it.
In the framework: `waitUntilLoaded()` in every page (ProductPage.java); unique last names in `TestData.newBooking()` (TestData.java); `RetryOnce` (RetryOnce.java).

Q50. How do you read a Playwright error message?
It states what was expected and received, the locator, the timeout, and a call log of every retry (e.g. *23 × locator resolved to … unexpected value "1"*). The call log shows whether the element was missing, hidden, or had the wrong text.
Example: `Locator expected to have text Expected: 2 Received: 1`.
In the framework: `cartBadgeCheckFails` produces exactly this in the report — ReportDemoTest.java.

7. Framework design

Q51. How does the Page Object Model look with Playwright?
Each page class holds its locators as `Locator` fields and offers user actions as methods; tests read like user stories and contain no selectors. Navigation methods return the next page object, so flows chain fluently.
Example:
`loginAs("standard_user").addToCart(BACKPACK).openCart().checkout().continueWith("Play", "Wright", "10115")`.
In the framework: the `pages` package — InventoryPage.java, used in CheckoutTest.java.

Q52. Do page objects need PageFactory or @FindBy?
No. Playwright locators are lazy, so plain fields created in the constructor work and never go stale. PageFactory is a Selenium concept.
Example: `private final Locator checkout = page.getByTestId("checkout");`.
In the framework: every page, e.g. CartPage.java.

Q53. Should page objects contain assertions?
Generally no: pages expose state (`Locator`s, values) and tests assert, so one page serves many checks. Waits that belong to the page itself (waiting until it is loaded) are the exception.
Example: `assertThat(inventory.cartBadge()).hasText("1")` in the test; `waitUntilLoaded()` inside the page.
In the framework: pages return `Locator`s like `error()`, `cartBadge()`; only `waitUntilLoaded()` uses `assertThat` — CheckoutInfoPage.java.

Q54. How do you share common parts like the header?
With a parent class for every page that shows them. Common helpers go into a base class for all pages.
Example: `ShopPage` holds the title, cart icon and side menu for all pages after login.
In the framework: `BasePage` → `ShopPage` → `InventoryPage`, `CartPage`, … — ShopPage.java, BasePage.java.

Q55. How do you manage configuration and environments?
A properties file with defaults, overridable by system properties (`-Dkey=value`) so CI or a person can change browser, URL or timeouts without editing code. Load it once (singleton) and fail with a clear message for a missing key.
Example: `mvn test -Dbrowser=firefox -DbaseUrl=https://staging.example.com`.
In the framework: Config.java + config.properties.

Q56. Where do test data live?
Static data and data providers in one class; dynamic data generated per test (unique names, future dates) so parallel runs and reruns never collide. Data a test creates in a shared system must be cleaned up.
Example: `TestData.newBooking()` with last name `Test` + 8 random digits.
In the framework: TestData.java.

Q57. How do you clean up test data?
Register what a test creates and delete it in an `@AfterMethod(alwaysRun = true)`, even when the test failed. Cleanup errors are logged, not thrown, so they never hide the real failure.
Example: `givenBooking(...)` adds the id to a list; the after-method deletes each id with

an admin token.
In the framework: `closeClient()` in ApiTest.java; UI tests need none, because each context is thrown away.

Q58. How do you make test steps visible in reports?
Log each user action from the page methods (not from tests), so every test gets readable steps for free. TestNG's `Reporter.log` attaches lines to the current test result.
Example: `Steps.log("Add \"Sauce Labs Backpack\" to the cart")`.
In the framework: Steps.java, called by every page method and by `ApiClient.send` (method, path, status, ms).

Q59. How do you avoid long, slow UI flows?
Test one behaviour per test, start each test as close to that behaviour as possible (API setup, saved login), and keep end-to-end flows for a few smoke tests.
Example: one full purchase test checks the maths; separate short tests check each required field.
In the framework: CheckoutTest.java; groups `smoke` mark the end-to-end ones.

8. TestNG core

Q60. Why TestNG with Playwright for Java?
Playwright for Java ships no test runner of its own (only JUnit 5 fixtures). TestNG adds groups, data providers, dependencies, parallel classes, listeners and suite files, which suit a UI + API framework well.
Example: `@Test(groups = "smoke", description = "...")`.
In the framework: TestNG 7.12 in pom.xml; suite in testng.xml.

Q61. What is the order of TestNG annotations?
`@BeforeSuite` → `@BeforeTest` → `@BeforeClass` → `@BeforeMethod` → `@Test` → `@AfterMethod` → `@AfterClass` → `@AfterTest` → `@AfterSuite`. Before/after-method run around every test method (and every data-provider row).
Example: open a context in `@BeforeMethod`, close it in `@AfterMethod`.
In the framework: `UiTest.openPage` / `closePage`, `ApiTest.openClient` / `closeClient`, `PlaywrightTest.closePlaywright` (`@AfterSuite`) — UiTest.java, PlaywrightTest.java.

Q62. Why alwaysRun = true on configuration methods?
Without it, a `@BeforeMethod` / `@AfterMethod` is skipped when groups are filtered (e.g. `-Dgroups=smoke`) and the method's class-level group does not match, and after a configuration failure. `alwaysRun` keeps setup and cleanup running.
Example: `@AfterMethod(alwaysRun = true) public void closePage(ITestResult`

`result)`.
In the framework: every before/after method in UiTest.java, ApiTest.java, PlaywrightTest.java.

Q63. How do groups work?
Groups tag tests (`smoke`, `api`); a class-level `@Test(groups = ...)` applies to all its methods and adds to method-level groups. Runs include or exclude groups in testng.xml or with Surefire's `groups` / `excludedGroups`.
Example: `mvn clean test -Dgroups=smoke`.
In the framework: `@Test(groups = "login")` on LoginTest.java + `groups = "smoke"` on chosen methods; `excludedGroups=demo` in pom.xml.

Q64. Hard vs soft assertions?
`Assert` stops the test at the first failure. `SoftAssert` records failures and throws them all at `assertAll()`; forgetting `assertAll()` makes the test pass silently.
Example: `soft.assertEquals(ping.status(), 201); soft.assertEquals(auth.status(), 200); soft.assertAll();`.
In the framework: hard asserts everywhere; `SoftAssert` in `softAssertChecksEverything` — TestNgFeaturesTest.java.

Q65. What do dependsOnMethods and priority do?
`priority` orders tests in a class (lower first). `dependsOnMethods` runs a test only after the named one passed and skips it otherwise, so one root cause shows as one failure plus skips.
Example: `@Test(priority = 2, dependsOnMethods = "apiIsUp")`.
In the framework: `apiIsUp` → `adminGetsATokenOnceTheApiIsUp` (TestNgFeaturesTest.java); the skip case in `skippedBecauseDependencyFailed` (ReportDemoTest.java).

Q66. How do you skip a test at runtime?
Throw `SkipException` with the reason; TestNG marks the test skipped (not failed). Use it for missing preconditions, not to hide failures.
Example: `throw new SkipException("Sauce Demo has no payment provider sandbox")`.
In the framework: `skippedWhenPreconditionIsMissing` — ReportDemoTest.java.

Q67. What do expectedExceptions, invocationCount and timeOut do?
`expectedExceptions` passes only if that exception is thrown (`expectedExceptionsMessageRegExp` also checks the message); `invocationCount` runs the method N times; `timeOut` fails a test that runs longer than N ms.
Example: `@Test(expectedExceptions = IllegalStateException.class, expectedExceptionsMessageRegExp = "No token from /auth.*")`.
In the framework: all three in TestNgFeaturesTest.java.

Q68. What is ITestResult used for?
It is the result of one test: status, name, parameters, timing, throwable, and attributes you can attach. An `@AfterMethod(ITestResult)` reads it to react to failures; listeners read attributes when writing reports.
Example: `if (result.getStatus() == ITestResult.FAILURE) ...;` `result.setAttribute("trace", path)`.
In the framework: `UiTest.closePage` attaches screenshot, video and trace; `HtmlReportListener` reads them — UiTest.java, HtmlReportListener.java.

9. Data providers, parallel runs, listeners

Q69. How does a DataProvider work?
A method annotated `@DataProvider(name = ...)` returns `Object[][]` (or an `Iterator<Object[]>`); each row runs the test once with those arguments. `dataProviderClass` points at a provider in another class (it must be static there).
Example: `@Test(dataProvider = "rejectedLogins", dataProviderClass = TestData.class)`.
In the framework: `rejectedLogins` ×5, `sortOptions` ×4, `incompleteCheckouts` ×3 in TestData.java; used by LoginTest.java.

Q70. How do you make data-provider rows readable in reports?
Put a case name as the first column and show it; TestNG's own reports list the parameters. Keep file names unique per row (e.g. add a timestamp) so artifacts never overwrite each other.
Example: `{ "locked-out user", LOCKED_OUT_USER, "secret_sauce", "Epic sadface: ..." }`.
In the framework: the report title adds the first parameter (*Login is rejected — wrong password*) — `renderTest` in HtmlReportListener.java; file names `Class_method_time` in UiTest.java.

Q71. Which parallel modes does TestNG have?
`methods`, `classes`, `tests` and `instances`, with a `thread-count`; data providers can run rows in parallel with `@DataProvider(parallel = true)`. `classes` keeps the methods of one class on one thread, which suits per-thread browsers.
Example: `<suite parallel="classes" thread-count="3">`.
In the framework: `parallel=classes`, `threadCount=${threads}` (3) in Surefire — pom.xml; the same in testng.xml.

Q72. How do you make a Playwright framework safe for parallel runs?
One Playwright per thread (`ThreadLocal`), one context per test, no static mutable state in tests, unique test data, and thread-safe collections in event handlers. Close every Playwright at the end (keep a list of all created).

Example: `private static final List<Playwright> ALL = new CopyOnWriteArrayList<>()`.
In the framework: BrowserManager.java; 3 threads, 54 tests in about 32 s on Chromium.

Q73. What is a listener, and which ones exist?
A class TestNG calls on events: `ITestListener` (test start/success/failure), `ISuiteListener`, `IInvokedMethodListener`, `IReporter` (after the run, for reports), `IRetryAnalyzer`, `IAnnotationTransformer` (changes annotations at load time).
Example: `public class HtmlReportListener implements IReporter`.
In the framework: HtmlReportListener.java builds the HTML report from all results.

Q74. How do you register a listener?
`@Listeners` on a class, `<listeners>` in testng.xml, Surefire's `listener` property, or Java's ServiceLoader file `META-INF/services/org.testng.ITestNGListener`. ServiceLoader works from Maven, the IDE and testng.xml alike; registering it twice writes the report twice.
Example: the services file contains `com.automation.playwright.report.HtmlReportListener`.
In the framework: META-INF/services/org.testng.ITestNGListener; testng.xml explains why it is not listed again.

Q75. How does retry work in TestNG?
`@Test(retryAnalyzer = X.class)`, where `X implements IRetryAnalyzer` and returns true from `retry(result)` to rerun. The failed attempt is reported as skipped. For all tests, set it through an `IAnnotationTransformer`.
Example: `if (retries < 1) { retries++; return true; }`.
In the framework: RetryOnce.java, used by `unknownBookingIsNotFoundWithRetry` in TestNgFeaturesTest.java; not global, so real failures are not hidden.

Q76. How are suite parameters passed?
`<parameter name="apiUser" value="admin"/>` in testng.xml plus `@Parameters("apiUser")` on the method; `@Optional("admin")` gives a default when no suite file is used (Maven without `suiteXmlFiles`).
Example: `public void suiteParameterUserGetsAToken(@Optional("admin") String user)`.
In the framework: testng.xml + TestNgFeaturesTest.java.

Q77. How do you rerun only the failed tests?
TestNG writes `testng-failed.xml` (the failed tests plus their configuration methods) to its output folder; run that file as a suite. Surefire's `-Dsurefire.rerunFailingTestsCount` works only for JUnit, not TestNG.
Example: `mvn test -Dsurefire.suiteXmlFiles=target/surefire-reports/testng-`

`failed.xml`.
In the framework: not scripted; after a `-Pdemo` run the file lists the demo failures in `target/surefire-reports/`.

10. Maven, CI, cross-browser and reports

Q78. How does Maven run TestNG tests?
The Surefire plugin finds `*Test` classes and runs them with TestNG when TestNG is on the classpath. It passes `groups`, `excludedGroups`, `parallel`, `threadCount`, and `-Dtest=Class#method` filters. With `suiteXmlFiles` set, it runs the suite file instead and ignores `-Dgroups` / `-Dtest`.
Example: `mvn clean test -Dtest=CheckoutTest -Dbrowser=webkit`.
In the framework: Surefire configuration without a suite file in pom.xml.

Q79. What is a Maven profile used for here?
A profile changes properties or plugins for one kind of run, switched on with `-P<id>`. It keeps optional behaviour out of the normal build.
Example: `mvn clean test -Pdemo` sets `excludedGroups=none`, so the demo tests run too.
In the framework: the `demo` profile in pom.xml.

Q80. Why did a path with spaces break the forked test JVM, and how was it fixed?
Surefire starts tests in a forked JVM with a manifest-only JAR on the classpath; some setups mis-handle spaces in paths there. `useSystemClassLoader=false` loads the test classpath through an isolated class loader instead.
Example: `<useSystemClassLoader>false</useSystemClassLoader>`.
In the framework: set with a comment in pom.xml.

Q81. How do you run Playwright tests in GitHub Actions?
Check out, set up a JDK with Maven caching, install the browser with `--with-deps` (Linux libraries), run the tests headless, and upload the report and failure artifacts with `if: always()` so they exist even when tests fail.
Example: `mvn -B -q compile exec:java -Dexec.args="install --with-deps chromium"` then `mvn -B test -Dbrowser=chromium`.
In the framework: tests.yml.

Q82. How do you run one suite on several browsers in CI?
A job matrix with one entry per browser; `fail-fast: false` lets the other browsers finish when one fails. Each job installs only its browser.
Example: `matrix: browser: [chromium, firefox, webkit]`.
In the framework: tests.yml; all three jobs green on the first run.

Q83. Can Playwright run in Docker?
Yes: Microsoft publishes `mcr.microsoft.com/playwright/java` images with JDK, Maven and browsers preinstalled; the CI job then skips the install step.
Example: `container: mcr.microsoft.com/playwright/java:v1.63.0-noble`.
In the framework: not used; `install --with-deps` on `ubuntu-latest` keeps the workflow simple.

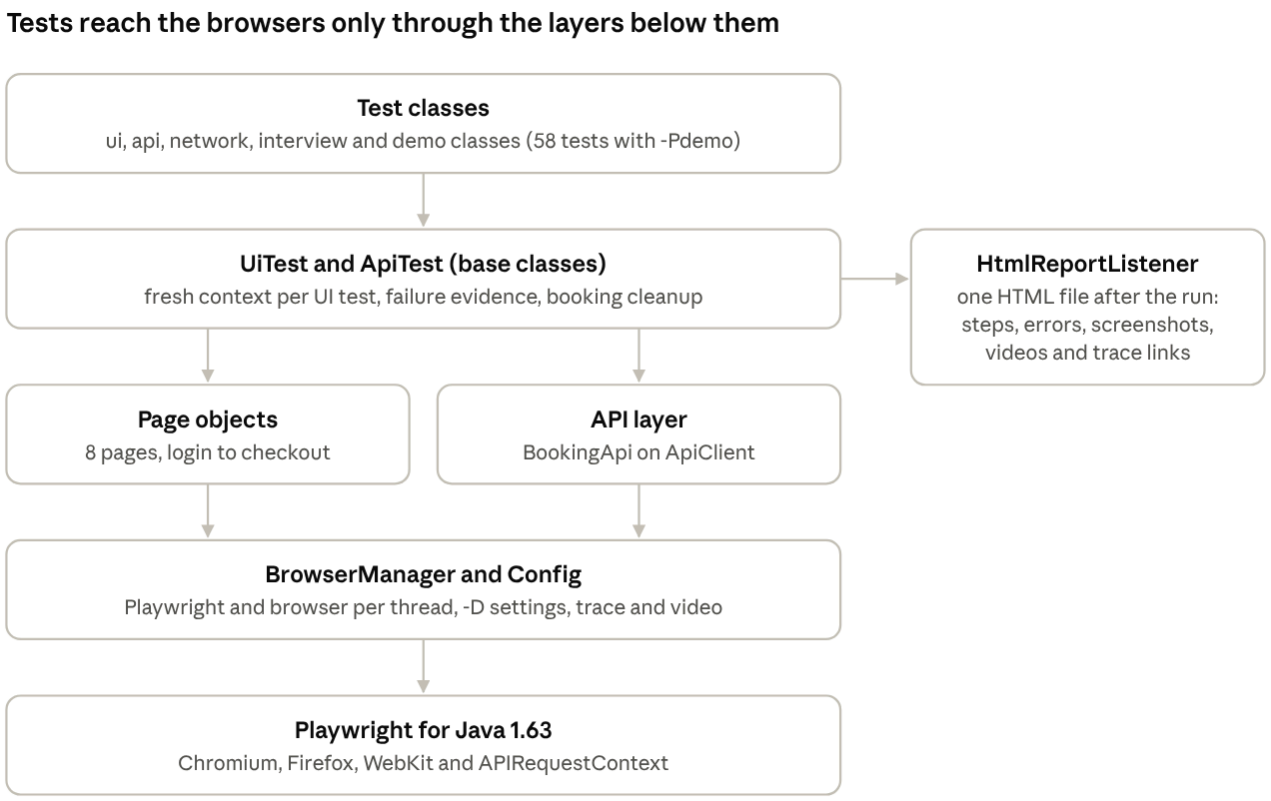
Q84. What did running three engines find?
Real differences: Firefox reports image downloads it cancels on navigation as failed requests (`NS_BINDING_ABORTED`), and WebKit showed the old page for a moment after the URL changed. Cross-browser runs catch timing assumptions that one engine hides.
Example: filter `abort` / `cancel` failures; wait for an element unique to the new page.
In the framework: NetworkTest.java, ProductPage.java; README “Latest results”.

Q85. Which reports does the project produce, and why a custom one?
Surefire writes XML/text reports and TestNG its own HTML; neither shows Playwright screenshots, videos or traces. A custom `IReporter` writes one self-contained HTML file with steps, errors, media and trace links. Allure or Extent are alternatives with plugins.
Example: `target/surefire-reports/playwright-test-report.html`.
In the framework: HtmlReportListener.java: class groups, filters, search, dark mode, intro animation (off for reduced motion).

Q86. How do you put screenshots into a single-file report?
Embed them as Base64 `data:image/png` URIs; videos and traces are too large, so they stay as files linked by a path relative to the report.
Example: `<img src="...">`.
In the framework: `UiTest.attachScreenshot` stores Base64 in a result attribute; `renderTest` embeds it and links video and trace with `link(reportDir, path)` — UiTest.java, HtmlReportListener.java.

11. About this project

Q87. Walk me through your framework.
Four layers. Config reads settings with `-D` overrides. `BrowserManager` gives each thread a Playwright and a browser, and each UI test a fresh context with trace and video. Page objects (UI) and `BookingApi` on `ApiClient` (API) hold all knowledge of the apps. Tests extend `UiTest` or `ApiTest`, which handle setup, failure evidence and data cleanup; a custom listener writes the HTML report.
Example: `LoginTest` → `UiTest.loginAs` → `LoginPage` → `BrowserManager.page()`.
In the framework: the layers are drawn below; README “Project structure”.



framework layers · tests to browsers

Each layer only calls the one below it; the base classes also hand screenshots, videos and traces to the report listener.

Q88. What does the suite cover?
54 tests in a normal run: UI 23 (login 9, inventory 6, cart 3, checkout 5), API 10 (auth 3, bookings 7), network 3, and 18 interview scenarios (Playwright 9, TestNG 9 runs). A `-Pdemo` run adds 2 failures and 2 skips on purpose, 58 in all.
Example: `mvn clean test -Dgroups=smoke` runs the end-to-end core.
In the framework: README “Tests” table; test classes under tests.

Q89. What was the hardest problem?
Tests that passed on Chromium failed on WebKit with a strict-mode violation: the URL had already changed but the old product list was still drawn. The fix was a `waitUntilLoaded()` per page that waits for an element only that page has, called by every navigation method; then all 3 engines passed.
Example: `ProductPage.waitUntilLoaded()` waits for the Back button.
In the framework: ProductPage.java, InventoryPage.java.

Q90. How does a failure look, and how do you analyse it?
The report opens the failing class first and shows the steps, the Playwright error with its call log, a full-page screenshot, a video of the whole test and the trace zip with the command to open it. Most failures are understood from the screenshot and call log; the

trace answers the rest.
Example: the demo `cartBadgeCheckFails` : expected 2, received 1, badge visible in the screenshot.
In the framework: ReportDemoTest.java; README screenshot `html-report-failure.png` .

Q91. How do you keep the shared demo API clean?
Every booking a test creates is registered and deleted after the test with an admin token, even when the test failed; a test that deletes its own booking tells the cleanup with `forget(id)` . Bookings get unique last names, so parallel tests never see each other's data.
Example: `int id = givenBooking(TestData.newBooking());` .
In the framework: ApiTest.java, BookingApiTest.java.

Q92. What would you add next?
Visual comparison for `visual_user` (screenshot diffs), an API-created state followed by a UI check on an app that shares them, a global retry through `IAnnotationTransformer` only if public-site outages become common, and publishing the report with GitHub Pages.
Example: `assertThat(page).hasScreenshot(...)` does not exist in Java; a pixel diff library (or the JS runner's `toHaveScreenshot`) would be needed.
In the framework: not built yet; listed under open items in the project notes.

12. Playwright coding scenarios

Each answer is a working test from PlaywrightScenariosTest.java, shortened here; all pass on Chromium, Firefox and WebKit.

Q93. Find the same field with five locator strategies and prove it is the same element.
Check that each locator has exactly one match and the same `data-test` value. Role and placeholder are the user-facing choices; CSS and XPath still work.

```
Map<String, Locator> strategies = Map.of(
        "getByTestId", page.getByTestId("username"),
        "getByPlaceholder", page.getByPlaceholder("Username"),
        "CSS", page.locator("#user-name"),
        "XPath", page.locator("//input[@name='user-name']"),
        "getByRole", page.getByRole(AriaRole.TEXTBOX, new
Page.GetByRoleOptions().setName("Username")));
strategies.forEach((name, locator) -> {
    assertThat(locator).hasCount(1);
    assertThat(locator).hasAttribute("data-test", "username");
});
```

In the framework: `locatorStrategiesFindTheSameField` .

Q94. Log in with the keyboard only.
Focus the first field, type, move on with Tab and submit with Enter, as a keyboard-only user would.

```
username.focus();
page.keyboard().type("standard_user");
page.keyboard().press("Tab");
page.keyboard().type("secret_sauce");
page.keyboard().press("Enter");
assertThat(page.getByTestId("inventory-item")).hasCount(6);
```

In the framework: `LoginPage.loginWithKeyboard` — LoginPage.java; test `loginWorksWithTheKeyboardOnly`.

Q95. Open a second tab and show that it shares the login.
Pages of one context share cookies, so the new tab opens the product list without logging in. Close it in `finally`.

```
Page second = page.context().newPage();
try {
    second.navigate("/inventory.html");
    assertThat(second.getByTestId("inventory-item")).hasCount(6);
    assertEquals(page.context().pages().size(), 2);
} finally {
    second.close();
}
```

In the framework: `secondTabSharesTheLogin`.

Q96. Save a login and reuse it in a new context.
`storageState()` returns cookies and local storage as JSON; a context created from it starts logged in. Try-with-resources closes the extra context.

```
String state = page.context().storageState();
try (BrowserContext context = page.context().browser().newContext(
        new
Browser.NewContextOptions().setBaseURL(baseUrl).setStorageState(state))) {
    Page fresh = context.newPage();
    fresh.navigate("/inventory.html");
    assertThat(fresh.getByTestId("inventory-item")).hasCount(6);
}
```

In the framework: `savedLoginSkipsTheLoginForm`.

Q97. Read page state and local storage with JavaScript.
`page.evaluate` runs a function in the page and returns its JSON-compatible result. Sauce Demo keeps the cart as product ids in local storage (the backpack is 4).

```
Object readyState = page.evaluate("() => document.readyState");
Object cart = page.evaluate("() => localStorage.getItem('cart-contents')");
assertEquals(readyState, "complete");
assertEquals(cart, "[4]");
```

In the framework: `javascriptReadsThePageAndItsStorage`.

Q98. Handle a JavaScript dialog.
Playwright dismisses dialogs automatically unless a handler is registered; `onceDialog` handles just the next one.

```
AtomicReference<String> message = new AtomicReference<>();
page.onceDialog(dialog -> {
    message.set(dialog.message());
    dialog.accept();
});
Object answer = page.evaluate("() => confirm('Leave the shop?')");
assertEquals(message.get(), "Leave the shop?");
assertEquals(answer, true);
```

In the framework: `browserDialogIsReadAndAccepted`.

Q99. Find broken images, and a response caused by an action.
`waitForResponse` wraps the action that triggers the request. To check every image, read the `src` values and request each one with `page.request()`, which shares the browser's cookies.

```
Response image = page.waitForResponse(r -> r.url().endsWith(".jpg"),
        () -> login.loginAs("standard_user"));
assertEquals(image.status(), 200);

for (String src : inventory.imageSources()) {
    APIResponse response = page.request().get(baseUrl + src);
    assertTrue(response.ok(), src + " answered " + response.status());
    response.dispose();
}
```

In the framework: `loginLoadsTheProductImages` and `productImagesAreNotBrokenExceptForTheProblemUser` (the `problem_user` sees one wrong picture on all 6 products).

13. TestNG coding scenarios

From TestNgFeaturesTest.java (on the Restful Booker API, no browser) and ReportDemoTest.java.

Q100. Check several things and report every failure at once.
`SoftAssert` collects failures; `assertAll()` throws them together.

```
SoftAssert soft = new SoftAssert();
soft.assertEquals(ping.status(), 201, "Status of /ping");
soft.assertEquals(auth.status(), 200, "Status of /auth");
soft.assertFalse(auth.json().path("token").asText("").isEmpty(), "A token
should be returned");
soft.assertAll();
```

In the framework: `softAssertChecksEverything`.

Q101. Test that a method throws the right exception with the right message.
The test passes only if `IllegalStateException` is thrown and its message matches the regex.

```
@Test(expectedExceptions = IllegalStateException.class,
      expectedExceptionsMessageRegExp = "No token from /auth.*Bad
credentials.*")
public void wrongPasswordThrowsTheExpectedException() {
    bookings.token("admin", "wrong-password");
}
```

In the framework: `wrongPasswordThrowsTheExpectedException`; the method under test is `BookingApi.token` — BookingApi.java.

Q102. Run tests in order and skip the next one if a precondition failed.
`priority` sets the order; `dependsOnMethods` skips the dependent test when the first fails.

```
@Test(priority = 1)
public void apiIsUp() { assertEquals(bookings.ping().status(), 201); }

@Test(priority = 2, dependsOnMethods = "apiIsUp")
public void adminGetsATokenOnceTheApiIsUp() {
assertFalse(adminToken().isEmpty()); }
```

In the framework: these two tests; the skip itself shows in `skippedBecauseDependencyFailed` in the demo run.

Q103. Retry a test once if it fails.
An `IRetryAnalyzer` keeps its own count; TestNG creates one per test method.

```
public class RetryOnce implements IRetryAnalyzer {
    private int retries;
    @Override
    public boolean retry(ITestResult result) {
        if (retries < 1) { retries++; return true; }
        return false;
    }
}

@Test(retryAnalyzer = RetryOnce.class)
public void unknownBookingIsNotFoundWithRetry() { ... }
```

In the framework: RetryOnce.java (also logs the retry as a report step).

Q104. Repeat a test, limit its time, and read a suite parameter.
`invocationCount` repeats, `timeOut` fails after the limit, `@Parameters` reads testng.xml with an `@Optional` default for Maven runs.

```
@Test(invocationCount = 2)
public void healthCheckAnswersEveryTime() {
assertEquals(bookings.ping().status(), 201); }

@Test(timeOut = 30_000)
public void tokenArrivesWithinThirtySeconds() {
assertFalse(adminToken().isEmpty()); }

@Parameters("apiUser")
@Test
public void suiteParameterUserGetsAToken(@Optional("admin") String user) {
    assertFalse(bookings.token(user, password).isEmpty());
}
```

In the framework: the three tests above; `<parameter name="apiUser" value="admin"/>` in testng.xml.

References

Where each file of the repository is used as an answer.

File	What it shows	Questions
BrowserManager.java	Playwright + browser per thread, context per test, trace, video, timeouts, test-id attribute	Q1, Q3, Q5, Q8, Q11, Q22, Q26, Q27, Q43, Q45, Q72
Config.java	Properties file with `-D` overrides	Q55
BasePage.java, ShopPage.java	Shared page helpers and header, role locators	Q4, Q10, Q16, Q54
LoginPage.java	`or()` wait with a clear failure, keyboard login, element screenshot	Q9, Q15, Q28, Q46, Q94
InventoryPage.java, ProductPage.java	Filter and chaining, CSS, `waitUntilLoaded()`	Q12–Q14, Q23, Q49, Q51, Q84, Q89
CartPage.java, checkout pages	Plain `Locator` fields, settled reads, no assertions in pages	Q18, Q20, Q52, Q53
ApiClient.java, BookingApi.java, Json.java	`APIRequestContext`, token cookie, records	Q38, Q39, Q41, Q101
HtmlReportListener.java	`IReporter`, embedded screenshots, trace links	Q44, Q68, Q70, Q73, Q85, Q86
Steps.java	Report steps via `Reporter.log`	Q58
UiTest.java, ApiTest.java, PlaywrightTest.java	Lifecycle, failure evidence, data cleanup	Q33, Q46, Q47, Q57, Q61, Q62, Q91
RetryOnce.java	`IRetryAnalyzer`	Q75, Q103
TestData.java	Data providers, generated bookings	Q56, Q69
UI tests (LoginTest.java, CartTest.java, CheckoutTest.java)	Web-first assertions, groups, data-provider tests	Q19, Q21, Q59, Q63
API tests (BookingApiTest.java)	CRUD, negative statuses	Q39, Q40
NetworkTest.java	Route, abort, fulfill, response events	Q34–Q36, Q84

File	What it shows	Questions
PlaywrightScenariosTest.java	Locator strategies, tabs, storage state, JS, dialogs, responses, images	Q12, Q29, Q30, Q37, Q42, Q93–Q99
TestNgFeaturesTest.java	SoftAssert, exceptions, order, repeats, time limit, retry, parameters	Q25, Q64, Q65, Q67, Q76, Q100–Q104
ReportDemoTest.java	Failures and skips on purpose	Q19, Q50, Q65, Q66, Q90
pom.xml, testng.xml	Surefire, parallel classes, demo profile, CLI, suite parameter	Q2, Q7, Q60, Q71, Q76, Q78–Q80
tests.yml	GitHub Actions matrix on three engines	Q3, Q7, Q81, Q82

Official documentation (opened 5 Oct 2026): Playwright for Java — Locators, Auto-waiting, Assertions, Browser contexts, Authentication, Network, API testing, Trace viewer, Videos, Multithreading, CI, Docker, Test runners; TestNG documentation; Maven Surefire — Using TestNG, test goal parameters.